

Advanced Models

IV

From MLP to CNN and RNN

Traditional neural networks are the multilayer perceptron architecture. It is built with fully-connected layers. This type of layers is simple in algorithm but with a lot of parameters. In more advanced application of deep learning, you will see other kind of architecture. In this chapter, you will be introduced to two popular architectures of deep learning. After you finished this chapter, you will learn

- ▷ What is convolutional neural network (CNN)
- ▷ What is recurrent neural network (RNN)
- ▷ Why CNN and RNN can do better in some applications

Let's get started.

Overview

This chapter is in two parts; they are:

- ▷ Introduction to Convolutional Neural Network
- ▷ Introduction to Recurrent Neural Network

24.1 Introduction to Convolutional Neural Network

Convolutional neural networks (CNNs) are deep neural networks that are designed to work with image data. They are commonly used for object recognition, image classification, and image segmentation. The key building blocks of a CNN are convolutional layers and pooling layers. Usually fully connected layers are also used as output layers in CNN models.

Convolutional layers are responsible for feature extraction by applying a set of learnable filters to the input image. Unlike the fully-connected layers, convolutional layers produce output with a sliding window which the filter is applied repeatedly. Filters are multiplied with different regions of the input image to identify the presence of specific features. Because filters are applied in a sliding window, the features are *spatially invariant* meaning a particular element in the image (e.g., a cat) is not required to appear at a specific location. Also, because of the filters are applied in a window, they are usually much smaller than the image.

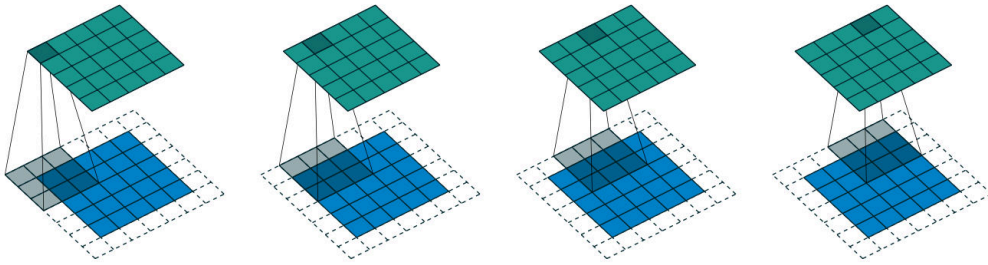


Figure 24.1: Operation of a convolutional layer: Blue is the input, gray is the filter, and green is the output. Image from “A guide to convolution arithmetic for deep learning”

Pooling layers in CNNs are to reduce the dimensionality of the feature maps generated by the convolutional layers. Similar to convolutional layers, it is applied with a sliding window. But instead of multiplying a filter, usually some aggregation function is used, such as taking the average or finding the maximum in the window. By carefully designing how the window slides, you can convert a high resolution image to lower resolution using pooling layer. This is how eventually you can reduce a million-pixel image into a binary classification (e.g., whether a cat is in the image), for example.

Fully-connected layers are used for classification by taking the output of the convolutional and pooling layers and passing it through a set of fully-connected neurons. Therefore, you usually see them near the output of the network model.

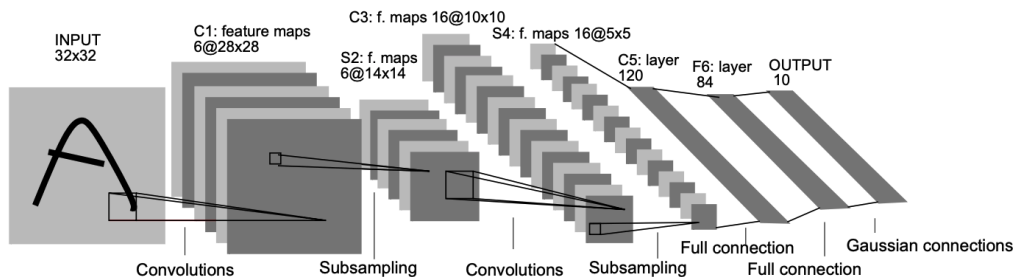


Figure 24.2: One of the earliest CNN, with convolutional layers, pooling layers (subsampling), and fully-connected layers. Image from “Gradient-Based Learning Applied to Document Recognition”

CNNs have been applied to a variety of image-related tasks such as object detection, facial recognition, and image segmentation. For example, it can be used to identify pedestrians, cars, and traffic signals on a street view to make a self-driving car.

CNNs are better suited for image-related tasks than Multilayer Perceptrons (MLPs) due to their ability to capture spatial information. MLPs process the input data as a flat vector, which means that they cannot take into account the spatial relationships between the different parts of an image. In contrast, CNNs keep the input image in two-dimensional and use convolutional layers to extract features from different regions of an image, allowing them to capture spatial information effectively.

Additionally, CNNs have fewer parameters than MLPs, which makes them more computationally efficient. MLPs require a large number of parameters to learn complex

patterns in data, whereas CNNs can learn the same patterns using fewer parameters. This makes CNNs more suitable for tasks that involve processing large amounts of data, such as image classification and object detection.

24.2 Introduction to Recurrent Neural Network

Recurrent neural networks (RNNs) are used for processing sequential data such as natural language text and speech. They are designed to capture the temporal dependencies between the input data.

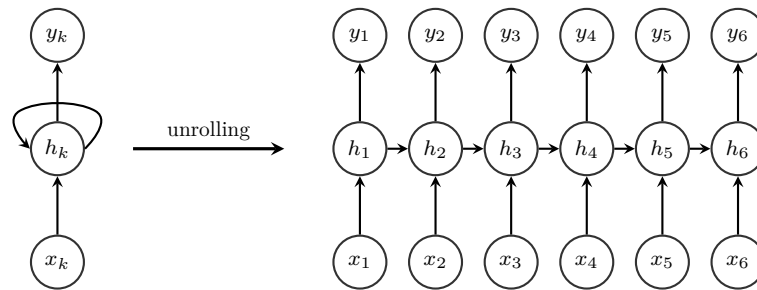


Figure 24.3: Structure of a recurrent neural network, before and after unrolling

The core building block of an RNN is the *recurrent layer*. The key idea behind a recurrent layer is to have one or more hidden state, which accumulates information about the input as the input sequence is provided. While the input is a sequence, the neural network is the same for each step, and produces output of the corresponding step. It is similar to an MLP processing one step of input at a time, except the hidden state is involved in producing the output and the input is indeed a sequence with many steps.

The hidden state of the recurrent layer is fed back into the layer at the next time step, allowing the network to maintain a memory of previous inputs. Therefore, sometimes you will see recurrent layer *unrolled* to emphasize the sequence structure of input and output. However, the repeating structure is indeed just one recurrent layer. The hidden state is not a learnable parameter, but rather, it is a state variable that is accumulated on the input sequence. It works as if it is the summary of the input sequence the RNN has seen so far. The processing of the input and the algorithm to update the hidden state is learned, but it is reused for every step of the input sequence. It is called *recurrent* because algorithmically, the recurrent layer is processing the input by applying the same algorithm in a loop on every step of the sequence.

RNNs can be more effective than Multilayer Perceptrons (MLPs) for sequential data tasks because they are designed to capture the temporal dependencies between the input data. MLPs process the input data as a flat vector, which means that they cannot take into account the order of the data. Moreover, since the input data is considered as a flat vector, inherently MLP is encouraged to process different time steps in the sequence differently. In contrast, RNNs use recurrent layers to maintain a memory of previous inputs, and consider each time step of the sequence as homogeneous, allowing them to capture the temporal dependencies in the data effectively.

The memory held by the hidden state makes RNNs well suited for tasks with variable length, such as language modeling, machine translation, and speech recognition. MLPs, on the other hand, require fixed-length inputs, which can be a limitation for tasks that involve variable-length data. In language processing, RNNs are used to generate, classify, and translate text. In speech recognition, RNN is used to convert spoken words into text, which RNN can analyze the sequence of sound waves produced by speech and identify the words being spoken.

24.3 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Vincent Dumoulin and Francesco Visin. “A guide to convolution arithmetic for deep learning”, 2016.

<https://arxiv.org/abs/1603.07285>

Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE*. Nov. 1998.

<http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf>

Andrej Karpathy. *The Unreasonable Effectiveness of Recurrent Neural Networks*. May 2015.

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Recurrent neural network. Wikipedia.

https://en.wikipedia.org/wiki/Recurrent_neural_network

24.4 Summary

CNN and RNN are two powerful architectures of deep learning models that are widely used in computer vision and natural language processing respectively. Understanding these architectures is essential for building effective deep learning models for a wide range of applications.

In the next chapter, you will learn to implement a CNN using PyTorch.

Building a Convolutional Neural Network in PyTorch

25

Neural networks are built with layers connected to each other. There are different kinds of layers. For image related applications, you can always find convolutional layers. It is a layer with very few parameters but applied over a large sized input. It is powerful because it can preserve the spatial structure of the image. Therefore, it is used to produce state-of-the-art results on computer vision neural networks. In this chapter, you will learn about the convolutional layer and the network it built. After completing this chapter, you will know:

- ▷ What are convolutional layers and pooling layers
- ▷ How they fit together in a neural network
- ▷ How a neural network using convolutional layers is designed

Let's get started.

Overview

This chapter is divided into four parts; they are

- ▷ The Case for Convolutional Neural Networks
- ▷ Building Blocks of Convolutional Neural Networks
- ▷ An Example of Convolutional Neural Network
- ▷ What Are in Feature Maps?

25.1 The Case for Convolutional Neural Networks

Let's consider making a neural network to process grayscale image as input, which is the simplest use case in deep learning for computer vision.

A grayscale image is an array of pixels. Each pixel is usually a value in a range of 0 to 255. An image with size 32×32 would have 1024 pixels. Taking it as input to a neural network would mean the first layer to have at least 1024 input weights.

Looking at the pixel value has little use to understanding the picture, because data is hiding in the spatial structure (e.g., whether there is a horizontal line or a vertical line on

the picture). Hence the traditional neural network will find it difficult to find out information from image input.

Convolutional neural network is to use convolutional layers to preserve spatial information of pixels. It learns how alike are the neighboring pixels and generating *feature representations*. What the convolutional layers see from the picture is invariant to distortion in some degree. For example, a convolutional neural network could predict the same result even if the input image has shift in color, rotated or rescaled. Moreover, convolutional layers have fewer weights, thus easier to train.

25.2 Building Blocks of Convolutional Neural Networks

The simplest use case of a convolutional neural network is for classification. You will find it to contain three types of layers:

1. Convolutional layers
2. Pooling layers
3. Fully-connected layers

Neurons on a convolutional layer is called the filter. Usually it is a 2D convolutional layer in image application. The filter is a 2D patch (e.g., 3×3 pixels) that is applied on the input image pixels. The size of this 2D patch is also called the receptive field, meaning how large a portion of the image it can see at a time.

The filter of a convolutional layer is to multiply with the input pixels, and then sum up the result. This result is one pixel value at the output. The filter will move around the input image to fill out all pixel values at the output. Usually multiple filters are applied to the same input, producing multiple output tensors. These output tensors are called the *feature maps* produced by this layer. They are stacked together as one tensor and pass on to the next layer as input.

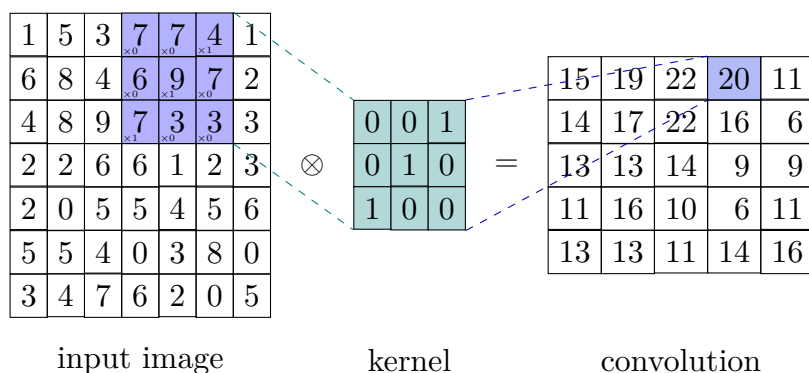


Figure 25.1: Example of a Filter Applied to a Two-Dimensional Input to Create a Feature Map

The output of a convolutional layer is called feature maps because usually it learned about the features of the input image. For example, whether there are vertical lines at the position. Learning the features from pixels is to help understanding the image at a higher

level. Multiple convolutional layers are stacked together in order to infer higher level features from lower level details.

Pooling layer is to *downsample* the previous layer's feature map. It is usually used after a convolutional layer to consolidate features learned. It can compress and generalize the feature representations. A pooling layer also has a receptive field and usually it is to take the average (average pooling) or the maximum (max pooling) over all values on the receptive field.

Fully connected layers are usually the final layers in a network. It is to take the features consolidated by previous convolutional and pooling layers as input to produce prediction. There might be multiple fully connected layers stacked together. In the case of classification, you usually see the output of the final fully connected layer applied with a softmax function to produce probability-like classification.

25.3 An Example of Convolutional Neural Network

The following is a program to do image classification on the CIFAR-10 dataset.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

transform = torchvision.transforms.Compose([torchvision.transforms.ToTensor()])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                       download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)

batch_size = 32
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size, shuffle=True)

class CIFAR10Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act1 = nn.ReLU()
        self.drop1 = nn.Dropout(0.3)

        self.conv2 = nn.Conv2d(32, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=(2, 2))

        self.flat = nn.Flatten()

        self.fc3 = nn.Linear(8192, 512)
        self.act3 = nn.ReLU()
        self.drop3 = nn.Dropout(0.5)
```

```

        self.fc4 = nn.Linear(512, 10)

    def forward(self, x):
        # input 3x32x32, output 32x32x32
        x = self.act1(self.conv1(x))
        x = self.drop1(x)
        # input 32x32x32, output 32x32x32
        x = self.act2(self.conv2(x))
        # input 32x32x32, output 32x16x16
        x = self.pool2(x)
        # input 32x16x16, output 8192
        x = self.flat(x)
        # input 8192, output 512
        x = self.act3(self.fc3(x))
        x = self.drop3(x)
        # input 512, output 10
        x = self.fc4(x)
        return x

model = CIFAR10Model()
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)

n_epochs = 20
for epoch in range(n_epochs):
    for inputs, labels in trainloader:
        # forward, backward, and then weight update
        y_pred = model(inputs)
        loss = loss_fn(y_pred, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    acc = 0
    count = 0
    for inputs, labels in testloader:
        y_pred = model(inputs)
        acc += (torch.argmax(y_pred, 1) == labels).float().sum()
        count += len(labels)
    acc /= count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

torch.save(model.state_dict(), "cifar10model.pth")

```

Listing 25.1: Classification model on CIFAR-10 image dataset

The CIFAR-10 dataset provides images in 32×32 pixels in RGB color (i.e., 3 color channels). There are 10 classes, labeled in integers 0 to 9. Whenever you are working on PyTorch neural network models for images, you will find the sister library `torchvision` useful. In the above, you used it to download the CIFAR-10 dataset from the Internet and transform it into a PyTorch tensor:

```
...
transform = torchvision.transforms.Compose([torchvision.transforms.ToTensor()])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                       download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
```

Listing 25.2: Load dataset and transform into PyTorch tensor

You also used a `DataLoader` in PyTorch to help creating batches for training. Training is to optimize the cross-entropy loss of the model, using stochastic gradient descent. It is a classification model, hence accuracy of classification is more intuitive than cross-entropy, which is computed at the end of each epoch, by comparing the maximum value in the output logit to the dataset's labels:

```
...
acc += (torch.argmax(y_pred, 1) == labels).float().sum()
```

Listing 25.3: Computing accuracy from logit output

It takes time to run the program above to train the network. This network should be able to achieve above 70% accuracy in classification.

It is typical in a network for image classification to be comprised of convolutional layers at an early stage, with dropout and pooling layers interleaved. Then, at a later stage, the output from convolutional layers is flattened and processed by some fully connected layers.

25.4 What Are in Feature Maps?

There are two convolutional layers in the network defined above. They are both defined with kernel size of 3×3 , hence it is looking at 9 pixels at a time to produce one output pixel. Note that the first convolutional layer is taking the RGB image as input. Hence each pixel has three channels. The second convolutional layer is taking a feature map with 32 channels as input. Each “pixel” as it sees will have 32 values. Thus the second convolutional layer has more parameters even they have the same receptive field.

Let's see what is in the feature map. Let's say we pick one input sample from the training set:

```
import matplotlib.pyplot as plt

plt.imshow(trainset.data[7])
plt.show()
```

Listing 25.4: Showing one image from the dataset

You should see that this is an image of a horse, in 32×32 pixels with RGB channels:

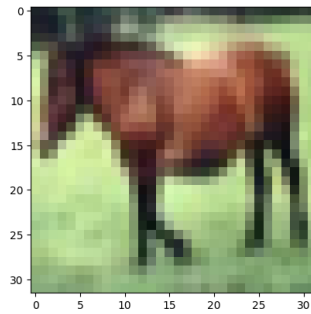


Figure 25.2: An example image

First, you need to convert this into a PyTorch tensor and make it a batch of one image. PyTorch models expect each image as a tensor in the format of (channel, height, width) but the data you read is in the format of (height, width, channel). If you use `torchvision` to transform the image into PyTorch tensors, this format conversion is done automatically. Otherwise, you need to *permute* the dimensions before use.

Afterward, pass it on through the model's first convolution layer and capture the output. You need to tell PyTorch that no gradient is needed for this calculation as you are not going to optimize the model weight:

```
X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)
model.eval()
with torch.no_grad():
    feature_maps = model.conv1(X)
```

Listing 25.5: Extracting feature map from a convolutional layer

The feature maps are in one tensor. You can visualize them using matplotlib:

```
fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()
```

Listing 25.6: Visualizing the feature map

You may see the following:

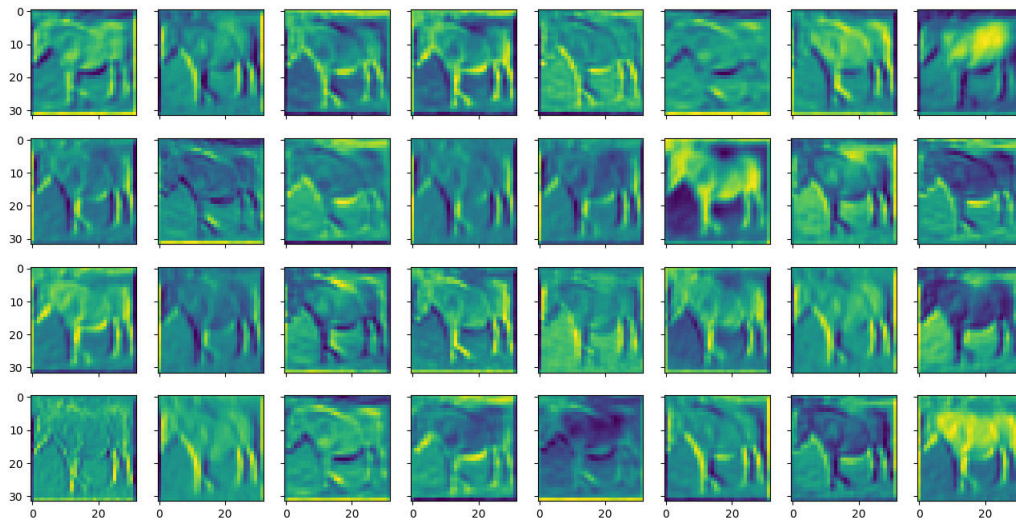


Figure 25.3: Feature map from an image

You can see that they are called feature maps because they are highlighting certain features from the input image. A feature is identified using a small window (in this case, over a 3×3 pixels filter). The input image has three color channels. Each channel has a different filter applied, and their results are combined for an output feature.

You can similarly display the feature map from the output of the second convolutional layer as follows:

```
X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)

model.eval()
with torch.no_grad():
    feature_maps = model.act1(model.conv1(X))
    feature_maps = model.drop1(feature_maps)
    feature_maps = model.conv2(feature_maps)

fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()
```

Listing 25.7: Extracting feature map from the second convolutional layer

Which shows:

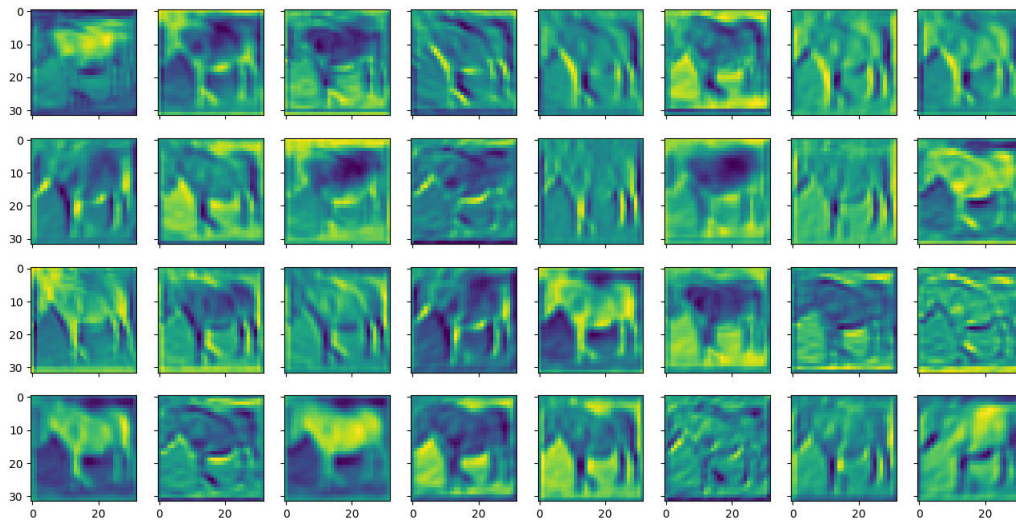


Figure 25.4: Feature map from the second convolutional layer

Compared to the output of the first convolutional layer, the feature maps from the second convolutional layer looks blurry and more abstract. But these are more useful for the model to identify the objects.

Putting everything together, the code below loads the saved model from the previous section and generate the feature maps:

```
import torch
import torch.nn as nn
import torchvision
import matplotlib.pyplot as plt

trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True)

class CIFAR10Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act1 = nn.ReLU()
        self.drop1 = nn.Dropout(0.3)

        self.conv2 = nn.Conv2d(32, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=(2, 2))

        self.flat = nn.Flatten()

        self.fc3 = nn.Linear(8192, 512)
        self.act3 = nn.ReLU()
        self.drop3 = nn.Dropout(0.5)

        self.fc4 = nn.Linear(512, 10)

    def forward(self, x):
```

```

        # input 3x32x32, output 32x32x32
        x = self.act1(self.conv1(x))
        x = self.drop1(x)
        # input 32x32x32, output 32x32x32
        x = self.act2(self.conv2(x))
        # input 32x32x32, output 32x16x16
        x = self.pool2(x)
        # input 32x16x16, output 8192
        x = self.flat(x)
        # input 8192, output 512
        x = self.act3(self.fc3(x))
        x = self.drop3(x)
        # input 512, output 10
        x = self.fc4(x)
        return x

model = CIFAR10Model()
model.load_state_dict(torch.load("cifar10model.pth"))

plt.imshow(trainset.data[7])
plt.show()

X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)
model.eval()
with torch.no_grad():
    feature_maps = model.conv1(X)
fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()

with torch.no_grad():
    feature_maps = model.act1(model.conv1(X))
    feature_maps = model.drop1(feature_maps)
    feature_maps = model.conv2(feature_maps)
fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()

```

Listing 25.8: Load a saved model and extract feature maps

25.5 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Vincent Dumoulin and Francesco Visin. “A guide to convolution arithmetic for deep learning”, 2016.

<https://arxiv.org/abs/1603.07285>

Training a Classifier. PyTorch tutorials.

https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

Conv2d. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Chapter 9, “Convolutional Networks”.

In: *Deep Learning*. MIT Press, 2016.

<https://www.amazon.com/dp/0262035618>

(Online version at <http://www.deeplearningbook.org>).

25.6 Summary

In this chapter, you learned how to use convolutional neural network to handle image input and how to visualize the feature maps.

Specifically, you learned:

- ▷ The structure of a typical convolutional neural networks
- ▷ What is the effect of the filter size to a convolutional layer
- ▷ What is the effect of stacking convolutional layers in a network
- ▷ How to extract and visualize feature maps from a convolutional neural network

In the next chapter, you will learn the classic example of using CNN to recognize handwritten digits.

Handwritten Digit Recognition with LeNet5 Model in PyTorch

26

A popular demonstration of the capability of deep learning techniques is object recognition in image data. The “hello world” of object recognition for machine learning and deep learning is the MNIST dataset for handwritten digit recognition. In this chapter, you will discover how to develop a deep learning model to achieve near state-of-the-art performance on the MNIST handwritten digit recognition task in PyTorch. After completing this chapter, you will know:

- ▷ How to load the MNIST dataset using torchvision
- ▷ How to develop and evaluate a baseline neural network model for the MNIST problem
- ▷ How to implement and evaluate a simple Convolutional Neural Network for MNIST
- ▷ How to implement a state-of-the-art deep learning model for MNIST

Let’s get started.

Overview

This chapter is divided into five parts; they are:

- ▷ The MNIST Handwritten Digit Recognition Problem
- ▷ Loading the MNIST Dataset in PyTorch
- ▷ Baseline Model with Multilayer Perceptrons
- ▷ Simple Convolutional Neural Network for MNIST
- ▷ LeNet5 for MNIST

26.1 The MNIST Handwritten Digit Recognition Problem

The MNIST problem is a classic problem that can demonstrate the power of convolutional neural networks. The MNIST dataset was developed by Yann LeCun, Corinna Cortes, and Christopher Burges for evaluating machine learning models on the handwritten digit classification problem. The dataset was constructed from a number of scanned document datasets available from the National Institute of Standards and Technology (NIST). This is where the name for the dataset comes from, the Modified NIST or MNIST dataset.

Images of digits were taken from a variety of scanned documents, normalized in size, and centered. This makes it an excellent dataset for evaluating models, allowing the developer to focus on machine learning with minimal data cleaning or preparation required. Each image is a 28×28 -pixel square (784 pixels total) in grayscale. A standard split of the dataset is used to evaluate and compare models, where 60,000 images are used to train a model, and a separate set of 10,000 images are used to test it.

The goal of this problem is to identify the digits on the image. There are ten digits (0 to 9) or ten classes to predict. The state-of-the-art prediction accuracy is at 99.8% level, achieved with large convolutional neural networks.

26.2 Loading the MNIST Dataset in PyTorch

The `torchvision` library is a sister project of PyTorch that provide specialized functions for computer vision tasks. There is a function in `torchvision` that can download the MNIST dataset for use with PyTorch. The dataset is downloaded the first time this function is called and stored locally, so you don't need to download again in the future. Below is a little script to download and visualize the first 16 images in the training subset of the MNIST dataset.

```
import matplotlib.pyplot as plt
import torchvision

train = torchvision.datasets.MNIST('./data', train=True, download=True)

fig, ax = plt.subplots(4, 4, sharex=True, sharey=True)
for i in range(4):
    for j in range(4):
        ax[i][j].imshow(train.data[4*i+j], cmap="gray")
plt.show()
```

Listing 26.1: Loading the MNIST dataset and display some samples

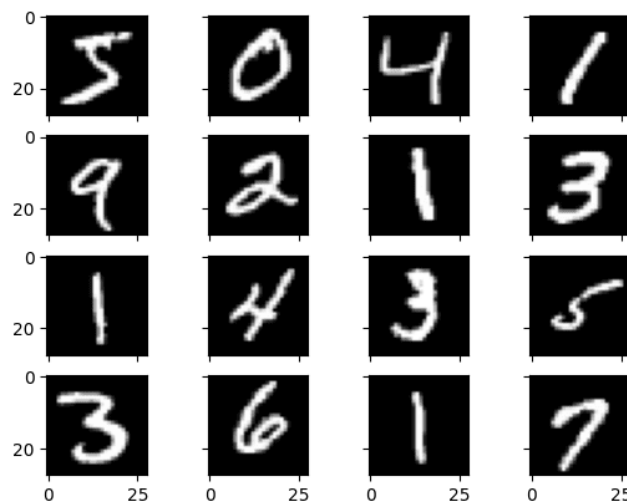


Figure 26.1: Examples from the MNIST dataset

26.3 Baseline Model with Multilayer Perceptrons

Do you really need a complex model like a convolutional neural network to get the best results with MNIST? You can get good results using a very simple neural network model with a single hidden layer. In this section, you will create a simple multilayer perceptron model that achieves accuracy of 99.81%. You will use this as a baseline for comparison to more complex convolutional neural network models. First, let's check what the data looks like:

```
import torchvision

# Load MNIST data
train = torchvision.datasets.MNIST('data', train=True, download=True)
test = torchvision.datasets.MNIST('data', train=False, download=True)
print(train.data.shape, train.targets.shape)
print(test.data.shape, test.targets.shape)
```

Listing 26.2: Showing the shape of the MNIST dataset

You should see:

```
torch.Size([60000, 28, 28]) torch.Size([60000])
torch.Size([10000, 28, 28]) torch.Size([10000])
```

Output 26.1: Shape of the MNIST dataset

The training dataset is structured as a 3-dimensional array of instance, image height, and image width. For a multilayer perceptron model, you must reduce the images down into a vector of pixels. In this case, the 28×28 -sized images will be 784 pixel input vectors. You can do this transform easily using the `reshape()` function.

The pixel values are grayscale between 0 and 255. It is almost always a good idea to perform some scaling of input values when using neural network models. Because the scale is well known and well behaved, you can very quickly normalize the pixel values to the range 0 and 1 by dividing each value by the maximum of 255.

In the following, you transform the dataset, convert to floating point, and normalize them by scaling floating point values and you can normalize them easily in the next step.

```
# each sample becomes a vector of values 0-1
X_train = train.data.reshape(-1, 784).float() / 255.0
y_train = train.targets
X_test = test.data.reshape(-1, 784).float() / 255.0
y_test = test.targets
```

Listing 26.3: Rescaling the dataset

The output targets `y_train` and `y_test` are labels in the form of integers from 0 to 9. This is a multiclass classification problem. You can convert these labels into one-hot encoding or keep them as integer labels like this case. You are going to use the cross-entropy function to evaluate the model performance and the PyTorch implementation of cross-entropy function can be applied on one-hot encoded targets or integer labeled targets.

You are now ready to create your simple neural network model. You will define your model in a PyTorch `Module` class.

```
class Baseline(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 784)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(784, 10)

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.layer2(x)
        return x
```

Listing 26.4: Baseline MLP model for classification of the MNIST dataset

The model is a simple neural network with one hidden layer with the same number of neurons as there are inputs (784). A rectifier activation function is used for the neurons in the hidden layer. The output of this model are *logits*, meaning they are real numbers which can be transformed into probability-like values using a softmax function. You do not apply the softmax function explicitly because the cross-entropy function will do that for you.

You will use the stochastic gradient descent algorithm (with learning rate set to 0.01) to optimize this model. The training loop is as follows:

```
model = Baseline()

optimizer = optim.SGD(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()
loader = torch.utils.data.DataLoader(list(zip(X_train, y_train)),
                                     shuffle=True, batch_size=100)

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    y_pred = model(X_test)
    acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
```

Listing 26.5: Training the baseline model with SGD

The MNIST dataset is small. This example should complete in a minute, with the output below. This simple network can produce accuracy at 92%.

```
Epoch 0: model accuracy 84.11%
Epoch 1: model accuracy 87.53%
Epoch 2: model accuracy 89.01%
Epoch 3: model accuracy 89.76%
Epoch 4: model accuracy 90.29%
Epoch 5: model accuracy 90.69%
Epoch 6: model accuracy 91.10%
Epoch 7: model accuracy 91.48%
Epoch 8: model accuracy 91.74%
Epoch 9: model accuracy 91.96%
```

Output 26.2: Progress of the model training

Below is the complete code for the above multilayer perceptron classification on MNIST dataset.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

# Load MNIST data
train = torchvision.datasets.MNIST('data', train=True, download=True)
test = torchvision.datasets.MNIST('data', train=False, download=True)

# each sample becomes a vector of values 0-1
X_train = train.data.reshape(-1, 784).float() / 255.0
y_train = train.targets
X_test = test.data.reshape(-1, 784).float() / 255.0
y_test = test.targets

class Baseline(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 784)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(784, 10)

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.layer2(x)
        return x

model = Baseline()

optimizer = optim.SGD(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()
loader = torch.utils.data.DataLoader(list(zip(X_train, y_train)),
                                     shuffle=True, batch_size=100)

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
```

```

        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    y_pred = model(X_test)
    acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Listing 26.6: Complete code of building MNIST classification baseline model

26.4 Simple Convolutional Neural Network for MNIST

Now that you have seen how to use multilayer perceptron model to classify MNIST dataset. Let's move on to try a convolutional neural network model. In this section, you will create a simple CNN for MNIST that demonstrates how to use all the aspects of a modern CNN implementation, including convolutional layers, pooling layers, and dropout layers.

In PyTorch, convolutional layers are supposed to work on images. Tensors for images should be the pixel values with the dimensions (sample, channel, height, width) but when you load images using libraries such as PIL, the pixels are usually presented as array of dimensions (height, width, channel). The conversion to a proper tensor format can be done using a transform from the torchvision library.

```

...
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0,), (128,)),
])
train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
test = torchvision.datasets.MNIST('data', train=False, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)

```

Listing 26.7: Creating a DataLoader for MNIST dataset

You need to use `DataLoader` because the transform is applied when you read the data from the `DataLoader`.

Next, define your neural network model. Convolutional neural networks are more complex than standard multilayer perceptrons, so you will start by using a simple structure that uses all the elements for state-of-the-art results. Below summarizes the network architecture.

1. The first hidden layer is a convolutional layer, `nn.Conv2d()`. The layer turns a grayscale image into 10 feature maps, with the filter size of 5×5 and a ReLU activation function. This is the input layer that expects images with the structure outlined above.
2. Next is a pooling layer that takes the max, `nn.MaxPool2d()`. It is configured with a pool size of 2×2 with stride 1. What it does is to take the maximum in a 2×2

pixel patch per channel and assign the value to the output pixel. The result is a 27×27 -pixels feature map per channel.

3. The next layer is a regularization layer using dropout, `nn.Dropout()`. It is configured to randomly exclude 20% of neurons in the layer in order to reduce overfitting.
4. Next is a layer that converts the 2D matrix data to a vector, using `nn.Flatten`. There are 10 channels from its input and each channel's feature map has size 27×27 . This layer allows the output to be processed by standard, fully connected layers.
5. Next is a fully connected layer with 128 neurons. ReLU activation function is used.
6. Finally, the output layer has ten neurons for the ten classes. You can transform the output into probability-like predictions by applying a softmax function on it.

This model is trained using cross-entropy loss and the Adam optimization algorithm. It is implemented as follows:

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 10, kernel_size=5, stride=1, padding=2)
        self.relu1 = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=1)
        self.dropout = nn.Dropout(0.2)
        self.flat = nn.Flatten()
        self.fc = nn.Linear(27*27*10, 128)
        self.relu2 = nn.ReLU()
        self.output = nn.Linear(128, 10)

    def forward(self, x):
        x = self.relu1(self.conv(x))
        x = self.pool(x)
        x = self.dropout(x)
        x = self.relu2(self.fc(self.flat(x)))
        x = self.output(x)
        return x

model = CNN()

optimizer = optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
```

```

acc = 0
count = 0
for X_batch, y_batch in testloader:
    y_pred = model(X_batch)
    acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
    count += len(y_batch)
acc = acc / count
print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Listing 26.8: Training a convolutional neural network model for MNIST

Running the above takes a few minutes and produces the following:

```

Epoch 0: model accuracy 81.74%
Epoch 1: model accuracy 85.38%
Epoch 2: model accuracy 86.37%
Epoch 3: model accuracy 87.75%
Epoch 4: model accuracy 88.00%
Epoch 5: model accuracy 88.17%
Epoch 6: model accuracy 88.81%
Epoch 7: model accuracy 88.34%
Epoch 8: model accuracy 88.86%
Epoch 9: model accuracy 88.75%

```

Output 26.3: Training progress of the CNN

Not the best result but this demonstrates how convolutional layer works. Below is the complete code for using the simple convolutional network.

```

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

# Load MNIST data
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0,), (128,)),
])
train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
test = torchvision.datasets.MNIST('data', train=False, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 10, kernel_size=5, stride=1, padding=2)
        self.relu1 = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=1)
        self.dropout = nn.Dropout(0.2)
        self.flat = nn.Flatten()
        self.fc = nn.Linear(27*27*10, 128)
        self.relu2 = nn.ReLU()

```

```

        self.output = nn.Linear(128, 10)

    def forward(self, x):
        x = self.relu1(self.conv(x))
        x = self.pool(x)
        x = self.dropout(x)
        x = self.relu2(self.fc(self.flat(x)))
        x = self.output(x)
        return x

model = CNN()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    acc = 0
    count = 0
    for X_batch, y_batch in testloader:
        y_pred = model(X_batch)
        acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
        count += len(y_batch)
    acc = acc / count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Listing 26.9: Complete code for classification using CNN

26.5 LeNet5 for MNIST

The previous model has only one convolutional layer. Of course, you can add more to make a deeper model. One of the earliest demonstration of the effectiveness of convolutional layers in neural networks is the “LeNet5” model. This model is developed to solve the MNIST classification problem. It has three convolutional layers and two fully connected layer to make up five trainable layers in the model, as it is named.

At the time it was developed, using hyperbolic tangent function as activation is common. Hence it is used here. This model is implemented as follows:

```

class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()

```

```

self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=2)
self.act1 = nn.Tanh()
self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)

self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
self.act2 = nn.Tanh()
self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)

self.conv3 = nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0)
self.act3 = nn.Tanh()

self.flat = nn.Flatten()
self.fc1 = nn.Linear(1*1*120, 84)
self.act4 = nn.Tanh()
self.fc2 = nn.Linear(84, 10)

def forward(self, x):
    # input 1x28x28, output 6x28x28
    x = self.act1(self.conv1(x))
    # input 6x28x28, output 6x14x14
    x = self.pool1(x)
    # input 6x14x14, output 16x10x10
    x = self.act2(self.conv2(x))
    # input 16x10x10, output 16x5x5
    x = self.pool2(x)
    # input 16x5x5, output 120x1x1
    x = self.act3(self.conv3(x))
    # input 120x1x1, output 84
    x = self.act4(self.fc1(self.flat(x)))
    # input 84, output 10
    x = self.fc2(x)
    return x

```

Listing 26.10: LeNet5 model implemented in PyTorch

Compare to the previous model, LeNet5 does not have Dropout layer (because Dropout layer was invented several years after LeNet5) and use average pooling instead of max pooling (i.e., for a patch of 2×2 pixels, it is taking average of the pixel values instead of taking the maximum). But the most notable characteristic of LeNet5 model is that it uses strides and paddings to reduce the image size from 28×28 pixel down to 1×1 pixel while increasing the number of channels from a one (grayscale) into 120.

Padding means to add pixels of value 0 at the border of the image to make it a bit larger. Without padding, the output of a convolutional layer will be smaller than its input. The stride parameter controls how much the filter should move to produce the next pixel in the output. Usually it is 1 to preserve the same size. If it is larger than 1, the output is a *downsampling* of the input. Hence you see in the LeNet5 model, stride 2 was used in the pooling layers to make, for example, a 28×28 -pixel image into 14×14 .

Training this model is same as training the previous convolutional network model, as follows:

```

...
model = LeNet5()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    acc = 0
    count = 0
    for X_batch, y_batch in testloader:
        y_pred = model(X_batch)
        acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
        count += len(y_batch)
    acc = acc / count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Listing 26.11: Training the LeNet5 model

Running this you may see:

```

Epoch 0: model accuracy 89.46%
Epoch 1: model accuracy 93.14%
Epoch 2: model accuracy 94.69%
Epoch 3: model accuracy 95.84%
Epoch 4: model accuracy 96.43%
Epoch 5: model accuracy 96.99%
Epoch 6: model accuracy 97.14%
Epoch 7: model accuracy 97.66%
Epoch 8: model accuracy 98.05%
Epoch 9: model accuracy 98.22%

```

Output 26.4: Progress of training the LeNet5 model

Here, we achieved accuracy beyond 98%.

The following is the complete code.

```

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

# Load MNIST data
transform = torchvision.transforms.Compose([

```

```

    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0,), (128,)),
])
train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
test = torchvision.datasets.MNIST('data', train=False, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)

class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=2)
        self.act1 = nn.Tanh()
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)

        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
        self.act2 = nn.Tanh()
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)

        self.conv3 = nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0)
        self.act3 = nn.Tanh()

        self.flat = nn.Flatten()
        self.fc1 = nn.Linear(1*1*120, 84)
        self.act4 = nn.Tanh()
        self.fc2 = nn.Linear(84, 10)

    def forward(self, x):
        # input 1x28x28, output 6x28x28
        x = self.act1(self.conv1(x))
        # input 6x28x28, output 6x14x14
        x = self.pool1(x)
        # input 6x14x14, output 16x10x10
        x = self.act2(self.conv2(x))
        # input 16x10x10, output 16x5x5
        x = self.pool2(x)
        # input 16x5x5, output 120x1x1
        x = self.act3(self.conv3(x))
        # input 120x1x1, output 84
        x = self.act4(self.fc1(self.flat(x)))
        # input 84, output 10
        x = self.fc2(x)
        return x

model = LeNet5()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)

```

```

        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    acc = 0
    count = 0
    for X_batch, y_batch in testloader:
        y_pred = model(X_batch)
        acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
        count += len(y_batch)
    acc = acc / count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Listing 26.12: Complete code of classification using LeNet5 model

26.6 Resources on MNIST

The MNIST dataset is very well studied. Below are some additional resources you might want to look into.

Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. *The MNIST database of handwritten digits*.

<http://yann.lecun.com/exdb/mnist/>

Rodrigo Benenson. *What is the class of this image?* Classification datasets results, 2016.

https://rodrigob.github.io/are_we_there_yet/build/classification_datasets_results.html

Digit Recognizer: Learn computer vision fundamentals with the famous MNIST data. Kaggle.

<https://www.kaggle.com/c/digit-recognizer>

Hubert Eichner. *Neural Net for Handwritten Digit Recognition in JavaScript*.

<http://myselfph.de/neuralNet.html>

Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE*. Nov. 1998.

<http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf>

Yann LeCun. *LeNet-5 convolutional neural networks*.

<http://yann.lecun.com/exdb/lenet/>

26.7 Summary

In this chapter, you discovered the MNIST handwritten digit recognition problem and deep learning models developed in Python using the PyTorch library that are capable of achieving excellent results. Working through this chapter, you learned:

- ▷ How to load the MNIST dataset in PyTorch with torchvision
- ▷ How to convert the MNIST dataset into PyTorch tensors for consumption by a convolutional neural network

- ▷ How to use PyTorch to create convolutional neural network models for MNIST
- ▷ How to implement the LeNet5 model for MNIST classification

In the next chapter, you will learn about LSTM, the most common type of recurrent neural network.

LSTM for Time Series Prediction in PyTorch

27

Long Short-Term Memory (LSTM) is a structure that can be used in neural network. It is a type of recurrent neural network (RNN) that expects the input in the form of a sequence of features. It is useful for data such as time series or string of text. In this chapter, you will learn about LSTM networks. In particular,

- ▷ What is LSTM and how they are different
- ▷ How to develop LSTM network for time series prediction
- ▷ How to train a LSTM network

Let's get started.

Overview

This chapter is divided into three parts; they are

- ▷ Overview of LSTM Network
- ▷ LSTM for Time Series Prediction
- ▷ Training and Verifying Your LSTM Network

27.1 Overview of LSTM Network

LSTM cell is a building block that you can use to build a larger neural network. While the common building block such as fully-connected layer are merely matrix multiplication of the weight tensor and the input to produce an output tensor, LSTM module is much more complex.

A typical LSTM cell is illustrated as follows

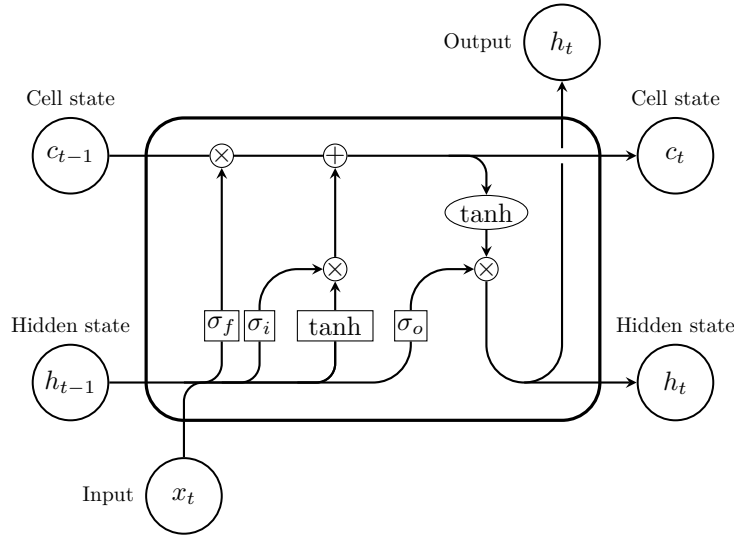


Figure 27.1: LSTM cell

It takes one time step of an input tensor x as well as a cell memory c and a hidden state h . The cell memory and hidden state can be initialized to zero at the beginning. Then within the LSTM cell, x , c , and h will be multiplied by separate weight tensors and pass through some activation functions a few times. The result is the updated cell memory and hidden state. These updated c and h will be used on the *next time step* of the input tensor. Until the end of the last time step, the output of the LSTM cell will be its cell memory and hidden state.

Specifically, the equation of one LSTM cell is as follows:

$$f_t = \sigma_f(W_f x_t + U_f h_{t-1} + b_f)$$

$$i_t = \sigma_i(W_i x_t + U_i h_{t-1} + b_i)$$

$$o_t = \sigma_o(W_o x_t + U_o h_{t-1} + b_o)$$

$$\tilde{c}_t = \sigma_c(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \otimes c_{t-1} + i_t \otimes \tilde{c}_t$$

$$h_t = o_t \otimes \sigma_h(c_t)$$

Where W , U , b are trainable parameters of the LSTM cell. Each equation above is computed for each time step, hence with subscript t . These trainable parameters are *reused* for all the time steps. This nature of shared parameter bring the memory power to the LSTM.

Note that the above is only one design of the LSTM. There are multiple variations in the literature.

Since the LSTM cell expects the input x in the form of multiple time steps, each input sample should be a 2D tensors: One dimension for time and another dimension for features. The power of an LSTM cell depends on the size of the hidden state or cell memory, which usually has a larger dimension than the number of features in the input.

27.2 LSTM for Time Series Prediction

Let's see how LSTM can be used to build a time series prediction neural network with an example.

The problem you will look at in this chapter is the international airline passengers prediction problem. This is a problem where, given a year and a month, the task is to predict the number of international airline passengers in units of 1,000. The data ranges from January 1949 to December 1960, or 12 years, with 144 observations.

It is a regression problem. That is, given the number of passengers (in unit of 1,000) the recent months, what is the number of passengers the next month. The dataset has only one feature: The number of passengers.

Let's start by reading the data. The data can be downloaded here.

▷ <https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv>

Save this file as `airline-passengers.csv` in the local directory for the following. Below is a sample of the first few lines of the file:

```
"Month","Passengers"  
"1949-01",112  
"1949-02",118  
"1949-03",132  
"1949-04",129
```

Output 27.1: The time series dataset in the CSV file

The data has two columns, the month and the number of passengers. Since the data are arranged in chronological order, you can take only the number of passenger to make a single-feature time series. Below you will use pandas library to read the CSV file and convert it into a 2D NumPy array, then plot it using matplotlib:

```
import matplotlib.pyplot as plt  
import pandas as pd  
  
df = pd.read_csv('airline-passengers.csv')  
timeseries = df[["Passengers"]].values.astype('float32')  
  
plt.plot(timeseries)  
plt.show()
```

Listing 27.1: Extracting and plotting the time series

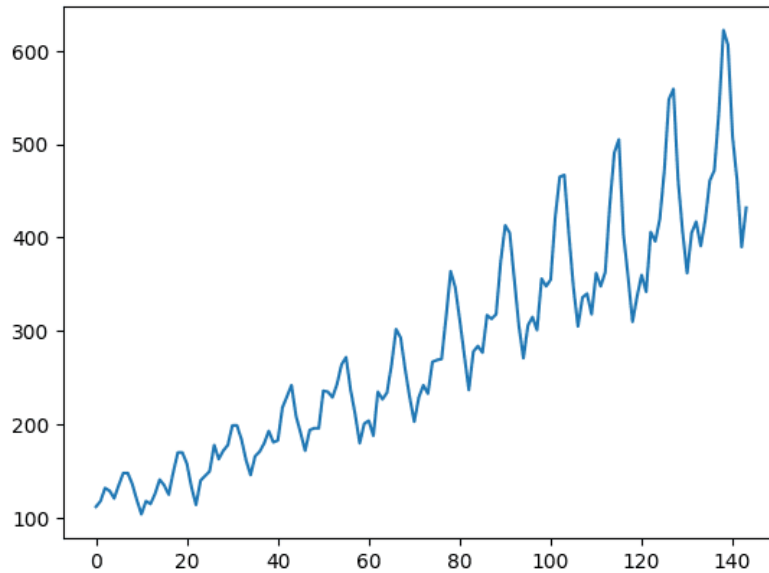


Figure 27.2: Time series as plotted

This time series has 144 time steps. You can see from the plot that there is an upward trend. There are also some periodicity in the dataset that corresponds to the summer holiday period in the northern hemisphere. Usually a time series should be “detrended” to remove the linear trend component and normalized before processing. For simplicity, these are skipped in this project.

To demonstrate the predictive power of our model, the time series is splitted into training and test sets. Unlike other dataset, usually time series data are splitted without shuffling. That is, the training set is the first half of time series and the remaining will be used as the test set. This can be easily done on a NumPy array:

```
# train-test split for time series
train_size = int(len(timeseries) * 0.67)
test_size = len(timeseries) - train_size
train, test = timeseries[:train_size], timeseries[train_size:]
```

Listing 27.2: Train-test split of the time series dataset

The more complicated problem is how do you want the network to predict the time series. Usually time series prediction is done on a window. That is, given data from time $t - w$ to time t , you are asked to predict for time $t + 1$ (or deeper into the future). The size of window w governs how much data you are allowed to look at when you make the prediction. This is also called the *look back period*.

On a long enough time series, multiple overlapping window can be created. It is convenient to create a function to generate a dataset of fixed window from a time series. Since the data is going to be used in a PyTorch model, the output dataset should be in PyTorch tensors:

```

import torch

def create_dataset(dataset, lookback):
    """Transform a time series into a prediction dataset

    Args:
        dataset: A numpy array of time series, first dimension is the time steps
        lookback: Size of window for prediction
    """
    X, y = [], []
    for i in range(len(dataset)-lookback):
        feature = dataset[i:i+lookback]
        target = dataset[i+1:i+lookback+1]
        X.append(feature)
        y.append(target)
    return torch.tensor(X), torch.tensor(y)

```

Listing 27.3: Transforming a time series dataset

This function is designed to apply windows on the time series. It is assumed to predict for one time step into the immediate future. It is designed to convert a time series into a tensor of dimensions (window sample, time steps, features). A time series of L time steps can produce roughly L windows (because a window can start from any time step as long as the window does not go beyond the boundary of the time series). Within one window, there are multiple consecutive time steps of values. In each time step, there can be multiple features. In this dataset, there is only one.

It is intentional to produce the *feature* and the *target* the same shape: For a window of three time steps, the feature is the time series from t to $t + 2$ and the target is from $t + 1$ to $t + 3$. What we are interested is $t + 3$ but the information of $t + 1$ to $t + 2$ is useful in training.

Note that the input time series is a 2D array and the output from the `create_dataset()` function will be a 3D tensor. Let's try with `lookback=1`. You can verify the shape of the output tensor as follows:

```

lookback = 1
X_train, y_train = create_dataset(train, lookback=lookback)
X_test, y_test = create_dataset(test, lookback=lookback)
print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)

```

Listing 27.4: Creating dataset tensor for training

which you should see:

```

torch.Size([95, 1, 1]) torch.Size([95, 1, 1])
torch.Size([47, 1, 1]) torch.Size([47, 1, 1])

```

Output 27.2: Shapes of tensors of the training and test set

Now you can build the LSTM model to predict the time series. With `lookback=1`, it is quite surely that the accuracy would not be good for too little clues to predict. But this is a good example to demonstrate the structure of the LSTM model.

The model is created as a class, in which a LSTM layer and a fully-connected layer is used.

```
...
import torch.nn as nn

class AirModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=50, num_layers=1, batch_first=True)
        self.linear = nn.Linear(50, 1)
    def forward(self, x):
        x, _ = self.lstm(x)
        x = self.linear(x)
        return x
```

Listing 27.5: LSTM model

The output of `nn.LSTM()` is a tuple. The first element is the generated hidden states, one for each time step of the input. The second element is the LSTM cell's memory and hidden states, which is not used here.

The LSTM layer is created with option `batch_first=True` because the tensors you prepared is in the dimension of (window sample, time steps, features) and where a batch is created by sampling on the first dimension.

The output of hidden states is further processed by a fully-connected layer to produce a single regression result. Since the output from LSTM is one per each input time step, you can choose to pick only the last timestep's output, which you should have:

```
x, _ = self.lstm(x)
# extract only the last time step
x = x[:, -1, :]
x = self.linear(x)
```

Listing 27.6: Modifying `forward()` to output only the next-step prediction

and the model's output will be the prediction of the next time step. But here, the fully connected layer is applied to each time step. In this design, you should extract only the last time step from the model output as your prediction. However, in this case, the window is 1, there is no difference in these two approaches.

27.3 Training and Verifying Your LSTM Network

Because it is a regression problem, MSE is chosen as the loss function, which is to be minimized by Adam optimizer. In the code below, the PyTorch tensors are combined into a dataset using `torch.utils.data.TensorDataset()` and batch for training is provided by a `DataLoader`. The model performance is evaluated once per 100 epochs, on both the training set and the test set:

```

import numpy as np
import torch.optim as optim
import torch.utils.data as data

model = AirModel()
optimizer = optim.Adam(model.parameters())
loss_fn = nn.MSELoss()
loader = data.DataLoader(data.TensorDataset(X_train, y_train), shuffle=True, batch_size=8)

n_epochs = 2000
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    if epoch % 100 != 0:
        continue
    model.eval()
    with torch.no_grad():
        y_pred = model(X_train)
        train_rmse = np.sqrt(loss_fn(y_pred, y_train))
        y_pred = model(X_test)
        test_rmse = np.sqrt(loss_fn(y_pred, y_test))
    print("Epoch %d: train RMSE %.4f, test RMSE %.4f" % (epoch, train_rmse, test_rmse))

```

Listing 27.7: Training a LSTM network

As the dataset is small, the model should be trained for long enough to learn about the pattern. Over these 2000 epochs trained, you should see the RMSE on both training set and test set decreasing:

```

Epoch 0: train RMSE 225.7571, test RMSE 422.1521
Epoch 100: train RMSE 186.7353, test RMSE 381.3285
Epoch 200: train RMSE 153.3157, test RMSE 345.3290
Epoch 300: train RMSE 124.7137, test RMSE 312.8820
Epoch 400: train RMSE 101.3789, test RMSE 283.7040
Epoch 500: train RMSE 83.0900, test RMSE 257.5325
Epoch 600: train RMSE 66.6143, test RMSE 232.3288
Epoch 700: train RMSE 53.8428, test RMSE 209.1579
Epoch 800: train RMSE 44.4156, test RMSE 188.3802
Epoch 900: train RMSE 37.1839, test RMSE 170.3186
Epoch 1000: train RMSE 32.0921, test RMSE 154.4092
Epoch 1100: train RMSE 29.0402, test RMSE 141.6920
Epoch 1200: train RMSE 26.9721, test RMSE 131.0108
Epoch 1300: train RMSE 25.7398, test RMSE 123.2518
Epoch 1400: train RMSE 24.8011, test RMSE 116.7029
Epoch 1500: train RMSE 24.7705, test RMSE 112.1551
Epoch 1600: train RMSE 24.4654, test RMSE 108.1879
Epoch 1700: train RMSE 25.1378, test RMSE 105.8224

```

```
Epoch 1800: train RMSE 24.1940, test RMSE 101.4219
Epoch 1900: train RMSE 23.4605, test RMSE 100.1780
```

Output 27.3: Training progress

It is expected to see the RMSE of test set is an order of magnitude larger. The RMSE of 100 means the prediction and the actual target would be in average off by 100 in value (i.e., 100,000 passengers in this dataset).

To better understand the prediction quality, you can indeed plot the output using matplotlib, as follows:

```
with torch.no_grad():
    # shift train predictions for plotting
    train_plot = np.ones_like(timeseries) * np.nan
    y_pred = model(X_train)
    y_pred = y_pred[:, -1, :]
    train_plot[lookback:train_size] = model(X_train)[:, -1, :]
    # shift test predictions for plotting
    test_plot = np.ones_like(timeseries) * np.nan
    test_plot[train_size+lookback:len(timeseries)] = model(X_test)[:, -1, :]
# plot
plt.plot(timeseries, c='b')
plt.plot(train_plot, c='r')
plt.plot(test_plot, c='g')
plt.show()
```

Listing 27.8: Plotting the predicted time series

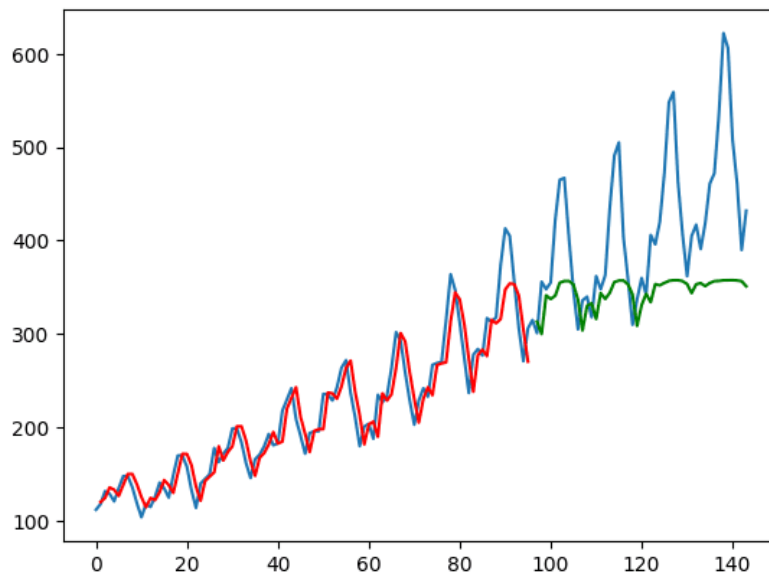


Figure 27.3: Predicted time series

From the above, you take the model's output as `y_pred` but extract only the data from the last time step as `y_pred[:, -1, :]`. This is what is plotted on the chart.

The training set is plotted in red while the test set is plotted in green. The blue curve is what the actual data looks like. You can see that the model can fit well to the training set but not very well on the test set.

Tying together, below is the complete code, except the parameter `lookback` is set to 4 this time:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data

df = pd.read_csv('airline-passengers.csv')
timeseries = df[["Passengers"]].values.astype('float32')

# train-test split for time series
train_size = int(len(timeseries) * 0.67)
test_size = len(timeseries) - train_size
train, test = timeseries[:train_size], timeseries[train_size:]

def create_dataset(dataset, lookback):
    """Transform a time series into a prediction dataset

    Args:
        dataset: A numpy array of time series, first dimension is the time steps
        lookback: Size of window for prediction
    """
    X, y = [], []
    for i in range(len(dataset)-lookback):
        feature = dataset[i:i+lookback]
        target = dataset[i+1:i+lookback+1]
        X.append(feature)
        y.append(target)
    return torch.tensor(X), torch.tensor(y)

lookback = 4
X_train, y_train = create_dataset(train, lookback=lookback)
X_test, y_test = create_dataset(test, lookback=lookback)

class AirModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=50, num_layers=1, batch_first=True)
        self.linear = nn.Linear(50, 1)
    def forward(self, x):
        x, _ = self.lstm(x)
        x = self.linear(x)
        return x

model = AirModel()
```

```

optimizer = optim.Adam(model.parameters())
loss_fn = nn.MSELoss()
loader = data.DataLoader(data.TensorDataset(X_train, y_train), shuffle=True, batch_size=8)

n_epochs = 2000
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    if epoch % 100 != 0:
        continue
    model.eval()
    with torch.no_grad():
        y_pred = model(X_train)
        train_rmse = np.sqrt(loss_fn(y_pred, y_train))
        y_pred = model(X_test)
        test_rmse = np.sqrt(loss_fn(y_pred, y_test))
    print("Epoch %d: train RMSE %.4f, test RMSE %.4f" % (epoch, train_rmse, test_rmse))

with torch.no_grad():
    # shift train predictions for plotting
    train_plot = np.ones_like(timeseries) * np.nan
    y_pred = model(X_train)
    y_pred = y_pred[:, -1, :]
    train_plot[lookback:train_size] = model(X_train)[:, -1, :]
    # shift test predictions for plotting
    test_plot = np.ones_like(timeseries) * np.nan
    test_plot[train_size+lookback:len(timeseries)] = model(X_test)[:, -1, :]
# plot
plt.plot(timeseries)
plt.plot(train_plot, c='r')
plt.plot(test_plot, c='g')
plt.show()

```

Listing 27.9: Training a LSTM model to predict time series with 4-step lookback

Running the above code will produce the plot below. From both the RMSE measure printed and the plot, you can notice that the model can now do better on the test set.

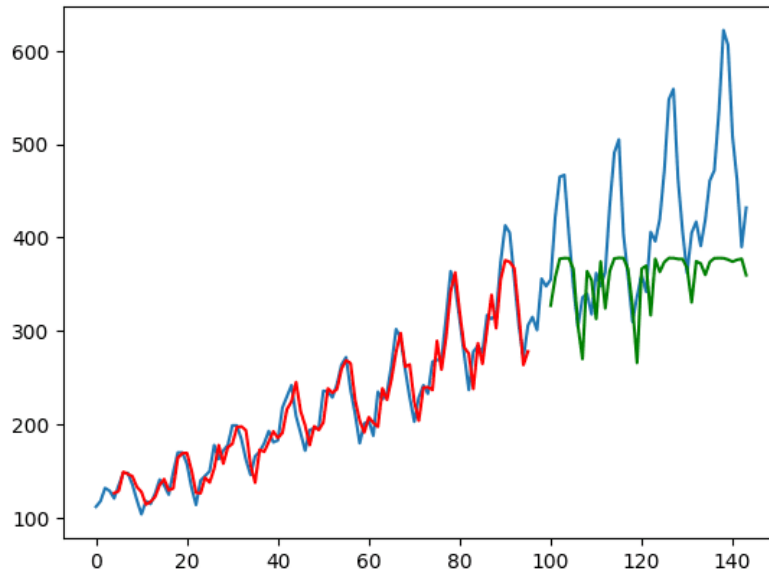


Figure 27.4: Time series prediction using LSTM network

This is also why the `create_dataset()` function is designed in such way: When the model is given a time series of time t to $t+3$ (as `lookback=4`), its output is the prediction of $t+1$ to $t+4$. However, $t+1$ to $t+3$ are also known from the input. By using these in the loss function, the model effectively was provided with more clues to train. This design is not always suitable, but you can see it is helpful in this particular example.

27.4 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

LSTM. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>

torch.utils.data. PyTorch documentation.

<https://pytorch.org/docs/stable/data.html>

27.5 Summary

In this chapter, you discovered what is LSTM and how to use it for time series prediction in PyTorch. Specifically, you learned:

- ▷ What is the international airline passenger time series prediction dataset
- ▷ What is a LSTM cell
- ▷ How to create an LSTM network for time series prediction

In the next chapter, you will see a larger example in LSTM.

Text Generation with LSTM in PyTorch

28

Recurrent neural network can be used for time series prediction. In which, a regression neural network is created. It can also be used as generative model, which usually is a classification neural network model. A generative model is to learn certain pattern from data, such that when it is presented with some prompt, it can create a complete output that in the same style as the learned pattern.

In this chapter, you will discover how to build a generative model for text using LSTM recurrent neural networks in PyTorch. After finishing this chapter, you will know:

- ▷ Where to download a free corpus of text that you can use to train text generative models
- ▷ How to frame the problem of text sequences to a recurrent neural network generative model
- ▷ How to develop an LSTM to generate plausible text sequences for a given problem

Let's get started.

Overview

This chapter is divided into six parts; they are:

- ▷ What is a Generative Model
- ▷ Getting Text Data
- ▷ A Small LSTM Network to Predict the Next Character
- ▷ Generating Text with an LSTM Model
- ▷ Using a Larger LSTM Network
- ▷ Faster Training with GPU

28.1 What is a Generative Model

Generative model is indeed, just another machine learning model that happened to be able to create new things. Generative Adversarial Network (GAN) is a class of its own. Transformer models that uses attention mechanism is also found to be useful to generate text passages.

It is just a machine learning model because the model has been trained with existing data, so that it learned something from it. Depends on how to train it, they can work vastly different. In this chapter, a character-based generative model is created. What it means is to train a model that take a sequence of characters (alphabets and punctuations) as input and the immediate next character as the target. As long as it can predict what is the next character given what are preceding, you can run the model in a loop to generate a long piece of text.

This model is probably the simplest one. However, human language is complex. You shouldn't expect it can produce very high quality output. Even so, you need a lot of data and train the model for a long time before you can see sensible results.

28.2 Getting Text Data

Obtaining high quality data is important for a successful generative model. Fortunately, many of the classical texts are no longer protected under copyright. This means you can download all the text for these books for free and use them in experiments, like creating generative models. Perhaps the best place to get access to free books that are no longer protected by copyright is Project Gutenberg.

In this chapter, you will use a favorite book from childhood as the dataset, Alice's Adventures in Wonderland by Lewis Carroll:

▷ <https://www.gutenberg.org/ebooks/11>

Your model will learn the dependencies between characters and the conditional probabilities of characters in sequences so that you can, in turn, generate wholly new and original sequences of characters. This chapter is a lot of fun, and repeating these experiments with other books from Project Gutenberg is recommended. These experiments are not limited to text; you can also experiment with other ASCII data, such as computer source code, marked-up documents in LATEX, HTML or Markdown, and more.

You can download the complete text in ASCII format (Plaintext UTF-8) for this book for free and place it in your working directory with the filename `wonderland.txt`. Now, you need to prepare the dataset ready for modeling. Project Gutenberg adds a standard header and footer to each book, which is not part of the original text. Open the file in a text editor and delete the header and footer. The header is obvious and ends with the text:

```
*** START OF THIS PROJECT GUTENBERG EBOOK ALICE'S ADVENTURES IN WONDERLAND ***
```

Output 28.1: Header of the text file

The footer is all the text after the line of text that says:

THE END

Output 28.2: Footer of the text file

You should be left with a text file that has about 3,400 lines of text.

28.3 A Small LSTM Network to Predict the Next Character

First, you need to do some preprocessing on the data before you can build a model. Neural network models can only work with numbers, not text. Therefore you need to transform the characters into numbers. To make the problem simpler, you also want to transform all uppercase letters into lowercase.

In below, you open the text file, transform all letters into lowercase, and create a Python dict `char_to_int` to map characters into distinct integers. For example, the list of unique sorted lowercase characters in the book is as follows:

```
[ '\n', '\r', ' ', '!', '"', "'", '(', ')', '*', ',', '-', '.', ':', ';', '?', '[', ']',
'_', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p',
'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', '\xbb', '\xbf', '\xef']
```

Listing 28.1: List of unique character tokens

Since this problem is character-based, the “vocabulary” are the distinct characters ever used in the text.

```
# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()

# create mapping of unique chars to integers
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))

# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)
```

Listing 28.2: Counting the vocabulary in the text file

This should print:

```
Total Characters: 144343
Total Vocab: 44
```

Output 28.3: Number of characters and vocabulary in the text file

You can see the book has just under 150,000 characters, and when converted to lowercase, there are only 44 distinct characters in the vocabulary for the network to learn — much more than the 26 in the alphabet.

Next, you need to separate the text into inputs and targets. A window of 100 character is used here. That is, with character 1 to 100 as input, your model is going to predict for character 101. Should a window of 5 be used, the word “chapter” will become two data samples:

```
chapt -> e
hapte -> r
```

Output 28.4: Input and prediction example

In a long text such as this one, a myriad of windows can be created and this produced a dataset of a lot of samples:

```
# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []
for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)
```

Listing 28.3: Creating a dataset from a long text using a sliding window

Running the above, you can see a total of 144,243 samples are created. Each sample is now in the form of integers, transformed using the mapping `char_to_int`. However, a PyTorch model would prefer to see the data in floating point tensors. Hence you should convert these into PyTorch tensors. LSTM layer is going to be used in the model, thus the input tensor should be of dimension (sample, time steps, features). To help training, it is also a good idea to normalize the input to 0 to 1. Hence you have the following:

```
import torch
import torch.nn as nn
import torch.optim as optim

# reshape X to be [samples, time steps, features]
X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)
X = X / float(n_vocab)
y = torch.tensor(dataY)
print(X.shape, y.shape)
```

Listing 28.4: Converting the dataset into PyTorch tensors

You can now define your LSTM model. Here, you define a single hidden LSTM layer with 256 hidden units. The input is single feature (i.e., one integer for one character). A dropout layer with probability 0.2 is added after the LSTM layer. The output of LSTM layer is a tuple, which the first element is the hidden states from the LSTM cell for each of the time step. It is a history of how the hidden state evolved as the LSTM cell accepts each time step of input. Presumably, the last hidden state contained the most information, hence only the

last hidden state is pass on to the output layer. The output layer is a fully-connected layer to produce logits for the 44 vocabularies. The logits can be converted into probability-like prediction using a softmax function.

```
import torch.nn as nn

class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=1, batch_first=True)
        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)
    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output
        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x
```

Listing 28.5: LSTM model for character prediction

This is a model for single character classification of 44 classes. Therefore cross-entropy loss should be used. It is optimized using Adam optimizer. The training loop is as follows. For simplicity, no test set has created, but the model is evaluated with the training set once again at the end of each epoch to keep track on the progress.

This program can run for a long time, especially on CPU! In order to preserve the fruit of work, the best model ever found is saved for future reuse.

```
n_epochs = 40
batch_size = 128
model = CharModel()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss(reduction="sum")
loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)

best_model = None
best_loss = np.inf
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    loss = 0
    with torch.no_grad():
        for X_batch, y_batch in loader:
```

```

        y_pred = model(X_batch)
        loss += loss_fn(y_pred, y_batch)
    if loss < best_loss:
        best_loss = loss
        best_model = model.state_dict()
    print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))

torch.save([best_model, char_to_dict], "single-char.pth")

```

Listing 28.6: Training loop for the LSTM network

Running the above may produce the following:

```

...
Epoch 35: Cross-entropy: 245745.2500
Epoch 36: Cross-entropy: 243908.7031
Epoch 37: Cross-entropy: 238833.5000
Epoch 38: Cross-entropy: 239069.0000
Epoch 39: Cross-entropy: 234176.2812

```

Output 28.5: Training progress on the LSTM network

The cross-entropy almost always decreasing in each epoch. This means probably the model is not fully converged and you can train it for more epochs. Upon the training loop completed, you should have the file `single-char.pth` created to contain the best model weight ever found, as well as the character-to-integer mapping used by this model.

For completeness, below is tying everything above into one script:

```

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data

# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()

# create mapping of unique chars to integers
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))

# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)

# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []

```

```

for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)

# reshape X to be [samples, time steps, features]
X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)
X = X / float(n_vocab)
y = torch.tensor(dataY)

class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=1, batch_first=True)
        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)
    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output
        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x

n_epochs = 40
batch_size = 128
model = CharModel()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss(reduction="sum")
loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)

best_model = None
best_loss = np.inf
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    loss = 0
    with torch.no_grad():
        for X_batch, y_batch in loader:
            y_pred = model(X_batch)
            loss += loss_fn(y_pred, y_batch)
    if loss < best_loss:
        best_loss = loss

```

```

best_model = model.state_dict()
print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))

torch.save([best_model, char_to_int], "single-char.pth")

```

Listing 28.7: Complete program to train a LSTM network for character prediction

28.4 Generating Text with an LSTM Model

Given the model is well-trained, generating text using the trained LSTM network is relatively straightforward. Firstly, you need to recreate the network and load the trained model weight from the saved checkpoint. Then you need to create some prompt for the model to start on. The prompt can be anything that the model can understand. It is a seed sequence to be given to the model to obtain one generated character. Then, the generated character is added to the end of this sequence, and trim off the first character to maintain the consistent length. This process is repeated for as long as you want to predict new characters (e.g., a sequence of 1,000 characters in length). You can pick a random input pattern as your seed sequence, then print generated characters as you generate them.

A simple way to generate prompt is to pick a random sample from the original dataset, e.g., with the `raw_text` obtained in the previous section, a prompt can be created as:

```

seq_length = 100
start = np.random.randint(0, len(raw_text)-seq_length)
prompt = raw_text[start:start+seq_length]

```

Listing 28.8: Sample a random prompt from the text

But you should be reminded that you need to transform it since this prompt is a string while the model expects a vector of integers.

The entire code is merely as follows:

```

import numpy as np
import torch
import torch.nn as nn

best_model, char_to_int = torch.load("single-char.pth")
n_vocab = len(char_to_int)
int_to_char = dict((i, c) for c, i in char_to_int.items())

# reload the model
class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=1, batch_first=True)
        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)
    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output

```

```

        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x
model = CharModel()
model.load_state_dict(best_model)

# randomly generate a prompt
filename = "wonderland.txt"
seq_length = 100
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()
start = np.random.randint(0, len(raw_text)-seq_length)
prompt = raw_text[start:start+seq_length]
pattern = [char_to_int[c] for c in prompt]

model.eval()
print('Prompt: "%s"' % prompt)
with torch.no_grad():
    for i in range(1000):
        # format input array of int into PyTorch tensor
        x = np.reshape(pattern, (1, len(pattern), 1)) / float(n_vocab)
        x = torch.tensor(x, dtype=torch.float32)
        # generate logits as output from the model
        prediction = model(x)
        # convert logits into one character
        index = int(prediction.argmax())
        result = int_to_char[index]
        print(result, end="")
        # append the new character into the prompt for the next iteration
        pattern.append(index)
        pattern = pattern[1:]
print()
print("Done.")

```

Listing 28.9: Loading the trained model and generate text from a random prompt

Running this example first outputs the prompt used, then each character as it is generated. For example, below are the results from one run of this text generator. The prompt was:

```
Prompt: "nother rush at the stick, and tumbled head
over heels in its hurry to get hold of it; then alice, th"
```

Output 28.6: The prompt

The generated text was:

```

e was got a litule soteet of thet was sh the thiee harden an the courd, and was tuitk
↳ a little toaee th thite ththe and said to the suher, and the whrtght the pacbit
↳ sese tha woode of the soeee, and the white rabbit ses ani thr gort to the thite
↳ rabbit, and then she was aoinnene th the three baaed of the sueen and saed "ota
↳ turpe "hun mot,"

```

```

"i don't know the ter ano _enend to mere," said the maccht ar a sore of great roaee.
↵ "ie you don't teink if thet soued to soeed to the boeie the mooer, io you bane
↵ thing it wo
tou het bn the crur,
"h whsh you cen not," said the manch hare.

"wes, it aadi," said the manch hare.

"weat you tail to merer ae in an a gens if gre" "he were thing," said the maccht ar a
↵ sore of geeaghen asd tothe to the thieg harden an the could.
"h dan tor toe taie thing," said the manch hare.

"wes, it aadi," said the manch hare.

"weat you tail to merer ae in an a gens if gre" "he were thing," said the maccht ar a
↵ sore of geeaghen asd tothe to the thieg harden an t

```

Output 28.7: The generated text

Let's note some observations about the generated text.

- ▷ It can emit line breaks. The original text limited the line width to 80 characters and the generative model attempted to replicate this pattern
- ▷ The characters are separated into word-like groups, and some groups are actual English words (e.g., "the," "said," and "rabbit"), but many are not (e.g., "thite," "soteet," and "tha").
- ▷ Some of the words in sequence make sense (e.g., "i don't know the"), but many do not (e.g., "he were thing").

The fact that this character-based model of the book produces output like this is very impressive. It gives you a sense of the learning capabilities of LSTM networks. However, the results are not perfect. In the next section, you will look at improving the quality of results by developing a much larger LSTM network.

28.5 Using a Larger LSTM Network

Recall that LSTM is a recurrent neural network. It takes a sequence as input, which in each step of the sequence, the input is mixed with its internal states to produce an output. Hence the output from LSTM is also a sequence. In the above, the output from the last time step is taken for further processing in the neural network but those from earlier steps are discarded. However, it is not necessarily the case. You can treat the sequence output from one LSTM layer as input to another LSTM layer. Then, you are building a larger network.

Similar to convolutional neural networks, a stacked LSTM network is supposed to have the earlier LSTM layers to learn low level features while the later LSTM layers to learn the high level features. It may not be always useful but you can try it out to see whether the model can produce a better result.

In PyTorch, making a stacked LSTM layer is easy. Let's modify the above model into the following:

```
import torch.nn as nn

class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=2,
                             batch_first=True, dropout=0.2)

        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)

    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output
        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x
```

Listing 28.10: A LSTM network with two LSTM layers

The only change is on the parameter to `nn.LSTM()`: you set `num_layers=2` instead of 1 to add another LSTM layer. But between the two LSTM layers, you also added a dropout layer through the parameter `dropout=0.2`. Replacing this model with the previous is all the change you need to make. Rerun the training you should see the below:

```
...
Epoch 34: Cross-entropy: 203763.0312
Epoch 35: Cross-entropy: 204002.5938
Epoch 36: Cross-entropy: 210636.5625
Epoch 37: Cross-entropy: 199619.6875
Epoch 38: Cross-entropy: 199240.2969
Epoch 39: Cross-entropy: 196966.1250
```

Output 28.8: Training progress of a larger LSTM network

You should see the cross-entropy here is lower than that in the previous section. This means this model is performing better. In fact, with this model, you can see the generated text looks more sensible:

```
Prompt: "ll
say that 'i see what i eat' is the same thing as 'i eat what i see'!"

"you might just as well sa"
y it to sea," she katter said to the jury. and the thoe hardeners vhine she was
↳ seady to alic the was a long tay of the sooe of the court, and she was seady to
↳ and taid to the coor and the court.
"well you see what you see, the mookee of the soog of the season of the shase of the
↳ court!"

"i don't know the rame thing is it?" said the caterpillar.

"the cormous was it makes he it was it taie the reason of the shall bbout it, you know."
```

```

"i don't know the rame thing i can't gelp the sea," the hatter went on, "i don't know
↳ the peally was in the shall sereat it would be a teally.
the mookee of the court "

"i don't know the rame thing is it?" said the caterpillar.

"the cormous was it makes he it was it taie the reason of the shall bbout it, you know."

"i don't know the rame thing i can't gelp the sea," the hatter went on, "i don't know
↳ the peally was in the shall sereat it would be a teally.
the mookee of the court "

"i don't know the rame thing is it?" said the caterpillar.

"the
Done.

```

Output 28.9: Generated text from a larger LSTM network

Not only words are spelled correctly, the text is also more English-like. Since the cross-entropy loss is still decreasing as you trained the model, you can assume the model is not converged yet. You can expect to make the model better if you increased the training epoch.

For completeness, below is the complete code for using this new model, including training and text generation.

```

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data

# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()

# create mapping of unique chars to integers
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))

# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)

# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []
for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]

```

```

    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)

# reshape X to be [samples, time steps, features]
X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)
X = X / float(n_vocab)
y = torch.tensor(dataY)

class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=2,
                             batch_first=True, dropout=0.2)
        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)
    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output
        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x

n_epochs = 40
batch_size = 128
model = CharModel()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss(reduction="sum")
loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)

best_model = None
best_loss = np.inf
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    loss = 0
    with torch.no_grad():
        for X_batch, y_batch in loader:
            y_pred = model(X_batch)
            loss += loss_fn(y_pred, y_batch)
        if loss < best_loss:
            best_loss = loss
            best_model = model.state_dict()
    print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))

```

```

torch.save([best_model, char_to_int], "single-char.pth")

# Generation using the trained model
best_model, char_to_int = torch.load("single-char.pth")
n_vocab = len(char_to_int)
int_to_char = dict((i, c) for c, i in char_to_int.items())
model.load_state_dict(best_model)

# randomly generate a prompt
filename = "wonderland.txt"
seq_length = 100
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()
start = np.random.randint(0, len(raw_text)-seq_length)
prompt = raw_text[start:start+seq_length]
pattern = [char_to_int[c] for c in prompt]

model.eval()
print('Prompt: "%s"' % prompt)
with torch.no_grad():
    for i in range(1000):
        # format input array of int into PyTorch tensor
        x = np.reshape(pattern, (1, len(pattern), 1)) / float(n_vocab)
        x = torch.tensor(x, dtype=torch.float32)
        # generate logits as output from the model
        prediction = model(x)
        # convert logits into one character
        index = int(prediction.argmax())
        result = int_to_char[index]
        print(result, end="")
        # append the new character into the prompt for the next iteration
        pattern.append(index)
        pattern = pattern[1:]
print()
print("Done.")

```

Listing 28.11: Complete code of training and using a larger LSTM network

28.6 Faster Training with GPU

Running programs from this chapter can be pathetically slow. Even if you have a GPU, you will not see immediate improvement. It is because the design of PyTorch, it may not use your GPU automatically. However, if you have a CUDA-capable GPU, you can improve the performance a lot by carefully moving the heavy computation away from your CPU.

A PyTorch model is a program of tensor calculation. The tensors can be stored in GPU or in CPU. Operation can be carried out as long as all the operators are in the same device. In this particular example, the model weight (i.e., those of the LSTM layers and the fully connected layer) can be moved to GPU. By doing so, the input should also be moved to the GPU before execution and the output will also be stored in the GPU unless you move it back.

In PyTorch, you can check if you have a CUDA-capable GPU using the following function:

```
torch.cuda.is_available()
```

Listing 28.12: Checking if CUDA is available on your computer

It returns a boolean to indicate if you can use GPU, which in turn, depends on the hardware model you have, whether your OS has the appropriate library installed, and whether your PyTorch is compiled with corresponding GPU support. If everything works in concert, you can create a device and assign your model to it:

```
device = torch.device("cuda:0")
model.to(device)
```

Listing 28.13: Sending a model to the GPU

If your model is running on CUDA device but your input tensor is not, you will see PyTorch complain about that and fail to proceed. To move your tensor to the CUDA device, you should run like the following:

```
y_pred = model(X_batch.to(device))
```

Listing 28.14: Sending a tensor to GPU and run on a GPU-based model

Which the `.to(device)` part will do the magic. But remember that `y_pred` produced above will also be on the CUDA device. Hence you should do the same when you run the loss function. Modifying the above program to make it capable to run on GPU will become the following:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data

# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()

# create mapping of unique chars to integers
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))

# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)

# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []
```

```

for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)

# reshape X to be [samples, time steps, features]
X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)
X = X / float(n_vocab)
y = torch.tensor(dataY)

class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=2,
                             batch_first=True, dropout=0.2)
        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)
    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output
        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x

n_epochs = 40
batch_size = 128
model = CharModel()
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model.to(device)

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss(reduction="sum")
loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)

best_model = None
best_loss = np.inf
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch.to(device))
        loss = loss_fn(y_pred, y_batch.to(device))
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    loss = 0
    with torch.no_grad():
        for X_batch, y_batch in loader:
            y_pred = model(X_batch.to(device))

```

```

        loss += loss_fn(y_pred, y_batch.to(device))
    if loss < best_loss:
        best_loss = loss
        best_model = model.state_dict()
    print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))

torch.save([best_model, char_to_int], "single-char.pth")

# Generation using the trained model
best_model, char_to_int = torch.load("single-char.pth")
n_vocab = len(char_to_int)
int_to_char = dict((i, c) for c, i in char_to_int.items())
model.load_state_dict(best_model)

# randomly generate a prompt
filename = "wonderland.txt"
seq_length = 100
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()
start = np.random.randint(0, len(raw_text)-seq_length)
prompt = raw_text[start:start+seq_length]
pattern = [char_to_int[c] for c in prompt]

model.eval()
print('Prompt: "%s"' % prompt)
with torch.no_grad():
    for i in range(1000):
        # format input array of int into PyTorch tensor
        x = np.reshape(pattern, (1, len(pattern), 1)) / float(n_vocab)
        x = torch.tensor(x, dtype=torch.float32)
        # generate logits as output from the model
        prediction = model(x.to(device))
        # convert logits into one character
        index = int(prediction.argmax())
        result = int_to_char[index]
        print(result, end="")
        # append the new character into the prompt for the next iteration
        pattern.append(index)
        pattern = pattern[1:]

print()
print("Done.")

```

Listing 28.15: Training and running a LSTM network on GPU if available

Compare to the code in the previous section, you should see they are essentially the same. Except the CUDA device is detected with the line:

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

Listing 28.16: Selecting the GPU if available and fallback to CPU otherwise

which will be your GPU or fall back to CPU if no CUDA device is found. Afterward, `.to(device)` is added at several strategic locations to move the computation to the GPU.

28.7 Further Readings

This character text model is a popular way of generating text using recurrent neural networks. Below are some more resources and tutorials on the topic if you are interested in going deeper.

Articles

Andrej Karpathy. *The Unreasonable Effectiveness of Recurrent Neural Networks*. May 2015.
<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Lars Eidnes. *Auto-Generating Clickbait With Recurrent Neural Networks*. 2015.
<https://larseidnes.com/2015/10/13/auto-generating-clickbait-with-recurrent-neural-networks/>

LSTM. PyTorch API.
<https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>

Sequence Models and Long Short-Term Memory Networks. PyTorch tutorials.
https://pytorch.org/tutorials/beginner/nlp/sequence_models_tutorial.html

Papers

Ilya Sutskever, James Martens, and Geoffrey Hinton. “Generating Text with Recurrent Neural Networks”. In: *Proceedings of the 28th International Conference on Machine Learning*. Bellevue, WA, USA, 2011.
<https://www.cs.utoronto.ca/~ilya/pubs/2011/LANG-RNN.pdf>

28.8 Summary

In this chapter, you discovered how you can develop an LSTM recurrent neural network for text generation in PyTorch. After completing this chapter, you know:

- ▷ How to find text for classical books for free as dataset for your machine learning model
- ▷ How to train an LSTM network for text sequences
- ▷ How to use a LSTM network to generate text sequences How to optimize deep learning training in PyTorch using CUDA devices

This marks the end of this book.